

2026-07-29T01-08-29

Tags:

micrograd notes

Outline

- Value class
 - Neural Network classes
 - Derivatives
 - Actual training loop
 - Forward pass
 - Gradients
 - Manual backpropagation
 - Operation-wise backward pass
 - Graph-wise backward pass
 - Accumulate gradients
 - Gradient descent optimization
 - Hyperparameters
-

Value class

```
import math

class Value:

    def __init__(self, data, _children=(), _op='', label=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(_children)
        self._op = _op
        self.label = label

    def __repr__(self):
        return f'Value(data={self.data})'

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), '+')

        def _backward():
            self.grad += 1.0 * out.grad
            other.grad += 1.0 * out.grad

        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), '*')

        def _backward():
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad
```

```

        out._backward = _backward
        return out

def __radd__(self, other):
    return self + other

def __rmul__(self, other):
    return self * other

def __pow__(self, other):
    assert isinstance(other, (int, float))
    out = Value(self.data**other, (self,), f'{self}**{other}')

    def _backward():
        local = other * (self.data ** (other-1))
        self.grad += local * out.grad

    out._backward = _backward
    return out

def __truediv__(self, other):
    return self * (other ** -1)

def __rtruediv__(self, other):
    return other * (self ** -1)

def __neg__(self):
    return self * -1

def __sub__(self, other):
    return self + (-other)

def __rsub__(self, other):
    return other + (-self)

def relu(self):
    out = Value(0 if self.data < 0 else self.data, (self,), 'ReLU')

    def _backward():
        self.grad += (out.data > 0) * out.grad
    out._backward = _backward

    return out

def tanh(self):
    x = self.data
    t = (math.exp(2*x) - 1)/(math.exp(2*x) + 1)
    out = Value(t, (self, ), 'tanh')

    def _backward():
        self.grad += (1 - (t ** 2.0)) * out.grad

    out._backward = _backward
    return out

def exp(self):
    x = self.data
    out = Value(math.exp(x), (self, ), 'exp')

    def _backward():
        self.grad += out.data * out.grad

    out._backward = _backward
    return out

def log(self):
    x = self.data

```

```

out = Value(log(x), (self, ), 'log')

def _backward():
    self.grad += (1/x) * out.grad

out._backward = _backward
return out

def backward(self):
    topo = []
    visited = set()

    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._prev:
                build_topo(child)
            topo.append(v)

    build_topo(self)

    self.grad = 1.0
    for node in reversed(topo):
        node._backward()

```

Neural Network classes

Neuron class

```

class Neuron:

    def __init__(self, nin):
        self.w = [Value(random.uniform(-1, 1)) for _ in range(nin)]
        self.b = Value(random.uniform(-1, 1))

    def __call__(self, x):
        # w * x + b
        # linear transform and activation
        z = sum((wi * xi for wi, xi in zip(self.w, x)), self.b)
        act = z.tanh()

        return act

    def parameters(self):
        return self.w + [self.b]

```

Layer class

```

class Layer:
    def __init__(self, nin, nout):
        self.neurons = [Neuron(nin) for _ in range(nout)]

    def __call__(self, x):
        outs = [n(x) for n in self.neurons]
        return outs[0] if len(outs) == 1 else outs

    def parameters(self):
        return [p for neuron in self.neurons for p in neuron.parameters()]

```

Multilayer Perception (MLP) class

```
class MLP:

    def __init__(self, nin, nouts):
        sz = [nin] + nouts
        self.layers = [Layer(sz[i], sz[i+1]) for i in range(len(nouts))]

    def __call__(self, x):
        for layer in self.layers:
            x = layer(x)

        return x

    def parameters(self):
        return [p for layer in self.layers for p in layer.parameters()]
```

Derivatives

The rate of change at a given point.

$$L = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}$$

Rules

Rule	$\frac{d}{dx}$
Power	nx^{n-1}
Product	$f'(x)g(x) + f(x)g'(x)$
Quotient	$\frac{f'(x) \cdot g(x) - f(x) \cdot g'(x)}{[g(x)]^2}$

Chain Rule

If z depends on y , and y depends on x

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

If we want $\frac{dL}{dc}$ and we know $\frac{dL}{dd}$ and $\frac{dd}{dc}$, then

$$\frac{dL}{dc} = \frac{dL}{dd} \cdot \frac{dd}{dc}$$

Also,

$$\begin{aligned} \frac{dL}{da} &= \frac{dL}{de} \cdot \frac{de}{da} \\ &= \frac{dL}{dd} \cdot \frac{dd}{de} \cdot \frac{de}{da} \end{aligned}$$

Basic derivatives

$f(x)$	$\frac{d}{dx}$
c	0
x	1
x^n	nx^{n-1}

- The derivative of a variable with respect to itself is (self derivative).

Exponential and logarithmic derivatives

$f(x)$	$\frac{d}{dx}$
e^x	e^x
a^x	$a^x \ln(a)$
$\ln(x)$	$\frac{1}{x}$

Trigonometric derivatives

$f(x)$	$\frac{d}{dx}$
$\sin(x)$	$\cos(x)$
$\cos(x)$	$-\sin(x)$
$\tan(x)$	$\sec^2(x)$

Derivatives for activation functions

$f(x)$	$\frac{d}{dx}$
$\tanh(x)$	$1 - \tanh^2(x)$
$\sigma(x) = \frac{1}{1 + e^{-x}}$	$\sigma(x)(1 - \sigma(x))$

Actual training loop

Input data and target labels

```
xs = [
    [2.0, 3.0, -1.0],
    [3.0, -1.0, 0.5],
    [0.5, 1.0, 1.0],
    [1.0, 1.0, -1.0],
]
ys = [1.0, -1.0, -1.0, 1.0] # desired targets
```

Training loop

```
# 20 epochs
for k in range(20):

    # 1. forward pass
    ypred = [n(x) for x in xs]
    loss = sum([(yout-ygt)**2 for ygt, yout in zip(ys, ypred)])

    # 2. backward pass
    for p in n.parameters():
        p.grad = 0.0 # zero the grad, commonly forgotten
    loss.backward()

    # 3. gradient descent update
    for p in n.parameters():
        lr = 0.05 # learning rate
        p.data += -lr * p.grad
```

```
print(k, loss.data, )
```

1. Forward pass
 - Loss function
 2. Backward pass
 3. Gradient descent optimization (update)
-

Forward pass

Forward pass (or propagation) computes the value of loss L .

! Remeber !

Each layer applies:

1. a linear transform z , and
2. a non-linear activation function a to a neuron.

For a layer l , each neuron will do:

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$
$$a^{[l]} = g^{[l]}(z^{[l]})$$

- W : weight matrix
- $a^{[l-1]}$: activation output of the previous layer
 - $a^{[0]} = x$ for input layer ($l = 1$)
- $a^{[l]}$: activation output of current layer
- b : bias vector

🧩 Example 🧩

Given:

```
x = [2.0, 3.0, -1.0]
n = MLP(3, [4, 4, 1])
n(x)
```

Where:

- `nin = 3`
- `nouts = [4, 4, 1]`
- `sz = [3, 4, 4, 1]`

At $l = 1$:

- `Layer(sz[0], sz[1]) = Layer(3, 4)`
- `Layer(3, 4) → 3 inputs, 4 outputs`
- 3 inputs →
 - each neuron has three input values.
 - each neuron has 3 weights and 1 bias.
- 4 outputs → 4 neurons are created for the layer.

At $l = 2$:

- `Layer(sz[1], sz[2]) = Layer(4, 4)`
- `Layer(4, 4) → 4 inputs, 4 outputs.`

At $l = 3$:

- `Layer(sz[2], sz[3]) = Layer(4, 1)`
- `Layer(4, 1) → 4 inputs, 1 output.`
- The final output of the network.

Loss function

Loss function measures the accuracy of the predictions.

? Why loss functions?

- We need a single measurement for total performance of a network → loss function.

Key relationship:

- Loss function (↓)
- Accuracy (↑)

Loss function	Shorthand	Formula
Mean squared error	MSE	$\frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$
Binary cross entropy	BCE	$-\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$

where:

- $y^{(i)}$ = true value, ground truth, target (`ygt`)
- $\hat{y}^{(i)}$ = predicted value, prediction (`yout`)

Loss	Easy formula
MSE	$(t - p)^2$
BCE	$-[t \log(p) + (1 - t) \log(1 - p)]$

where:

- t = target, true value
- p = prediction

Gradients

Continuing on from [derivatives](#).

`grad` means the derivative of an output (i.e., L) with respect to a `Value`.

Analytical gradient	Numerical gradient
Use derivative identities.	Use derivative definition

```

def f(a, b, c):
    return -a**3 + sin(3*b) - 1.0/c + b**2.5 - a**0.5

# Analytical
def gradf(a, b, c):
    wrt_a = -3*(a**2) - (1/2)*a**(-1/2)
    wrt_b = 3*cos(3*b) + (5/2)*b**(3/2)
    wrt_c = 1/(c**2)

    # return [df/da, df/db, df/dc]
    return [wrt_a, wrt_b, wrt_c]

analytical_grad = gradf(2, 3, 4)

# Numerical
def derive(f, x):
    h = 0.000001
    numer = (f(x+h)-f(x))
    denum = h

    return numer/denum

wrt_a = derive(lambda x: f(x, 3, 4), 2)
wrt_b = derive(lambda x: f(2, x, 4), 3)
wrt_c = derive(lambda x: f(2, 3, x), 4)

numerical_grad = [wrt_a, wrt_b, wrt_c]

# Numerical (Symmetrical)
# better approximation
def sym_derive(f, x):
    h = 0.000001
    numer = (f(x+h)-f(x-h))
    denum = 2*h

    return numer/denum

wrt_a = sym_derive(lambda x: f(x, 3, 4), 2)
wrt_b = sym_derive(lambda x: f(2, x, 4), 3)
wrt_c = sym_derive(lambda x: f(2, 3, x), 4)

numerical_grad2 = [wrt_a, wrt_b, wrt_c]

```

A symmetric derivative provides a better approximation.

$$L = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a-h)}{2h}$$

Manual backpropagation

Backpropagation is simply a recursive application of [chain rule](#). It computes how *sensitive* is L to every variable.

Given the following manual forward pass:

```

# inputs x1, x2
x1 = Value(2.0, label='x1')
x2 = Value(0.0, label='x2')

# weights w1, w2
w1 = Value(-3.0, label='w1')
w2 = Value(1.0, label='w2')

```

```

# bias
b = Value(6.8813735870195432, label='b')

# x1*w1
x1w1 = x1 * w1
x1w1.label = 'x1*w1'

# x2*w2
x2w2 = x2 * w2
x2w2.label = 'x2*w2'

# x1*w1 + x2*w2
x1w1x2w2 = x1w1 + x2w2
x1w1x2w2.label = 'x1*w1 + x2*w2'

# x1*w1 + x2*w2 + b
n = x1w1x2w2 + b
n.label = 'n'

# activation
o = n.tanh()
o.label = 'o'

```

The manual backproagation is given step-by-step below:

```

# 1. self-derivative is always 1
#  $\partial o / \partial o = 1$ 
o.grad = 1.0

```

```

# o = tanh(n)
# 2.  $\partial o / \partial n = \partial o / \partial o$ 
#           =  $\partial o / \partial [\tanh(n)]$ 
#           =  $1 - [\tanh(n)]^2$ 
n.grad = 1 - (o.data ** 2)

```

```

# 3. + is a distributor of grad
#  $\partial o / \partial (x1w1 + x2w2) = \partial o / \partial n * \partial n / \partial (x1w1 + x2w2)$ 
#           =  $\partial o / \partial n * 1$ 
#  $\partial o / \partial b = \partial o / \partial n * \partial n / \partial b$ 
#           =  $\partial o / \partial n * 1$ 
x1w1x2w2.grad = n.grad * 1
b.grad = n.grad * 1

```

```

# 4. + again
#  $\partial o / \partial (x1w1) = \partial o / \partial (x1w1 + x2w2) * \partial (x1w1 + x2w2) / \partial (x1w1)$ 
#           =  $\partial o / \partial (x1w1 + x2w2) * 1$ 
#
#  $\partial o / \partial (x2w2) = \partial o / \partial (x1w1 + x2w2) * \partial (x1w1 + x2w2) / \partial (x2w2)$ 
#           =  $\partial o / \partial (x1w1 + x2w2) * 1$ 
x1w1.grad = x1w1x2w2.grad * 1
x2w2.grad = x1w1x2w2.grad * 1

```

```

# 5. * is a switcher of grad
#  $\partial o / \partial x1 = \partial o / \partial (x1w1) * \partial (x1w1) / \partial x1$ 
#           =  $\partial o / \partial (x1w1) * w1$ 
#
#  $\partial o / \partial w1 = \partial o / \partial (x1w1) * \partial (x1w1) / \partial w1$ 
#           =  $\partial o / \partial (x1w1) * x1$ 
x1.grad = x1w1.grad * w1.data
w1.grad = x1w1.grad * x1.data

```

```
# 6. again
#  $\partial o / \partial x2 = \partial o / \partial (x2w2) * \partial (x2w2) / \partial x2$ 
#           =  $\partial o / \partial (x2w2) * w2$ 
#
#  $\partial o / \partial w2 = \partial o / \partial (x2w2) * \partial (x2w2) / \partial w2$ 
#           =  $\partial o / \partial (x2w2) * x2$ 
x2.grad = x2w2.grad * w2.data
w2.grad = x2w2.grad * x2.data
```

! Remember !

- Self-derivative is always 1.
- `+` is a distributor of `grad`.
- `*` is a switcher of `grad`.

Operation-wise backward pass

As seen in `Value` class:

- `self._backward` stores the local chain-rule function that propagates output gradient to the input gradients.
- For leaf nodes, it defaults to `lambda: None` because there's nothing to propagate.

Given the forward pass from [Manual backpropagation](#) and `self._backward` from `Value`:

```
o.grad = 1.0
o._backward()
n._backward()
b._backward() # leaf node, nothing happens
x1w1x2w2._backward()
x1w1._backward()
x2w2._backward()
```

Graph-wise backward pass

Using topological sort, we can do a graph-wise backward pass.

- *Topological sort*: a traversal where a node `v` is only visited after all dependencies are visited.
- Dependencies first, dependent node second.

From the `Value` class:

```
def backward(self):
    topo = []
    visited = set()

    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._prev:
                build_topo(child)
            topo.append(v)

    build_topo(self)

    self.grad = 1.0
```

```
for node in reversed(topo):
    node._backward()
```

! Remember !

- Reset gradients to 0 before each backward pass.
- Computed gradients should only represent the current forward pass, NOT the accumulated gradients from previous iterations.

```
for p in n.parameters():
    p.grad = 0.0
loss.backward()
```

Accumulate gradients

What if:

```
a = Value(3.0)
b = a + a
```

Because $b = a + a$, then $\frac{\partial b}{\partial a} = 2$ (`a.grad = 2.0`).

! Remeber !

- Multiple paths (even from the same variable) must have their gradients added together
- Use `+=` when calculating for `self.grad` on nested `_backward()` .

Gradient descent optimization

In gradient descent:

- Gradient is a vector pointing to the direction of **INCREASED** loss.
- Modify parameter data (`p.data`) by a small step size in the direction of the gradient.

	Meaning	What to do?
positive gradient	+ parameter → + loss	decrease parameter
negative gradient	+ parameter → - loss	increase parameter

```
for p in n.parameters():
    lr = 0.05 # learning rate
    p.data += -lr * p.grad
```

Learning
rate

- A hyperparameter
- Controls how large each parameter update should be

Step
size

- The actual amount a parameter changes during an update

Hyperparameters

Parameters

- Learned automatically during training.

Hyperparameters

- Set before training.
- Controls training.
- Indirectly affects parameters.

Examples of parameters:

- $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}, \dots$

Examples of hyperparameters:

- learning rate α
- epochs
- no. of iterations
- no. of hidden layers L
- no. of hidden units $n^{[1]}, n^{[2]}, \dots$
- choice of activation functions (ReLU, sigmoid, etc.)
- momentum term, minibatch size, regularization parameters, etc.